1a) BIT STUFFING

```
#include <stdio.h>
#include <string.h>
int main()
{
   int i, j = 0, count = 0;
   char s[100], d[100];
   printf("Enter a string: ");
   scanf("%s", s);
   int a = strlen(s);
   for (i = 0; i < a; i++)
   {
      d[j++] = s[i];
      if (s[i] == '1')
      {
         count++;
         if (count == 5)
         {
            d[j++] = '0';
            count = 0;
         }
      }
      else
      {
         count = 0;
      }
   }
```

```
    d[j] = '\0';
    printf("After bit stuffing: %s\n", d);
    return 0;
}
```

1b) CHARACTER STUFFING

```
#include <stdio.h>
#include <string.h>
int main()
{
    int i = 0, j = 0, m;
    char a[20], b[50], ch;
    printf("Enter String\n");
    scanf("%s", a);
    getchar();
    printf("Enter the character\n");
    ch = getchar();
    m = strlen(a);
    b[0] = 'd';
    b[1] = 'l';
    b[2] = 'e';
    b[3] = 's';
    b[4] = 't';
    b[5] = 'x';
    j = 6;
    while (i < m)
    {
```

```
        if (i + 2 < m && a[i] == 'd' && a[i+1] == 'l' && a[i+2] == 'e')
        {
            b[j] = 'd';
            b[j+1] = 'l';
            b[j+2] = 'e';
            j = j + 3;
        }
        b[j] = a[i];
        i++;
        j++;
    }
    b[j] = 'd';
    b[j+1] = 'l';
    b[j+2] = 'e';
    b[j+3] = 'e';
    b[j+4] = 't';
    b[j+5] = 'x';
    b[j+6] = '\0';
    printf("\nFrame after stuffing:\n");
    printf("%s", b);
    printf("\nFrame after destuffing:\n");
    printf("%s", a);
}
```

1c) CHARACTER COUNT

```
#include <stdio.h>
#include <string.h>
char data[20][20];
```

```
int n;
int main()
{
   int i, j, chi;
   char tmp[10][20];
   printf("Enter the number of frames:");
   scanf("%d", &n);
   for (i = 0; i < n; i++)
   {
      printf("\nframe %d:\n", i);
      scanf("%s", data[i]);
   }
   for (i = 0; i < n; i++)
   {
      tmp[i][0] = 49 + strlen(data[i]);
      tmp[i][1] = '0';
      tmp[i][2] = '0';
      tmp[i][3] = '\0';
      strcat(tmp[i], data[i]);
   }
   printf("\n\nAT THE SENDER\n");
   printf("\nData as frames:\n");
   for (i = 0; i < n; i++)
   {
      printf("frame : %d ", i);
      puts(tmp[i]);
   }
```

```
    printf("\nData transmitted:\n");
    for (i = 0; i < n; i++)
    {
        printf("\n%s", tmp[i]);
    }
    printf("\n\nAT THE RECEIVER\n");
    printf("\nThe data received:\n");
    chi = (int)(tmp[0][0] - 49);
    for (j = 1; j <= chi; j++)
    {
        data[0][j - 1] = tmp[0][j + 2];
    }
    data[0][chi] = '\0';
    printf("\nThe data after Removing count char:\n");
    for (i = 0; i < n; i++)
    {
        printf("\n%s", data[i]);
    }
    printf("\nThe data in frame form:\n");
    for (i = 0; i < n; i++)
    {
        printf("frame : %d ", i);
        puts(data[i]);
    }
}
```

2) CRC

```
#include <stdio.h>
```

```
#include <string.h>
char t[100], cs[100], g[20];
int a, i, N;
void XOR()
{
    for (int c = 0; c < N; c++)
        cs[c] = (cs[c] == g[c]) ? '0' : '1';
}
void crc() {
    int e = 0;
    for (int c = 0; c < N; c++)
        cs[c] = t[c];
    e = N;
    while (e <= a + N - 1) {
        if (cs[0] == '1')
            XOR();
        for (int c = 0; c < N - 1; c++)
            cs[c] = cs[c + 1];
        cs[N - 1] = t[e];
        e++;
    }
}
int main() {
    int choice;
    printf("Enter data: ");
    scanf("%s", t);
    printf("Enter Divisor: ");
```

```
scanf("%s", g);
a = strlen(t);
N = strlen(g);
for (int i = a; i < a + N - 1; i++)
    t[i] = '0';
t[a + N - 1] = '\0';
printf("Modified data is: %s\n", t);
crc();
printf("Remainder is: %s\n", cs);
for (int i = 0; i < N - 1; i++)
    t[a + i] = cs[i];
t[a + N - 1] = '\0';
printf("Final codeword is: %s\n", t);
printf("Test error detection? (0 for yes, 1 for no): ");
scanf("%d", &choice);
if (choice == 0) {
    char received[100];
    printf("Enter Received codeword: ");
    scanf("%s", received);
    strcpy(t, received);
    crc();
    int error = 0;
    for (int i = 0; i < N - 1; i++) {
        if (cs[i] != '0') {
            error = 1;
            break;
        }
```

```
    }
    printf("Remainder after checking received codeword: %s\n", cs);

    if (error)
        printf("Error detected in received codeword\n");
    else
        printf("No error detected in received codeword\n");
  }
  return 0;
}
```

3) SLIDING WINDOW

```
#include <stdio.h> #include <stdlib.h> #include <time.h>

#define ll long long int

void transmission(ll *i, ll N, ll t) { while (*i < t) { int z = 0;
    for (ll x = *i; x < *i + N && x < t; x++) {
        printf("Sending frame: %lld\n", x);
    }

    for (ll x = *i; x < *i + N && x < t; x++) {
        int f = rand() % 2;
        if (f == 1) {
            printf("Acknowledgment for frame: %lld\n", x);
            z++;
        } else {
            printf("Time out: frame number %lld not received\n", x);
            printf("Retransmitting window:\n");
            break;
        }
    }

    printf("\n");
```

```
    *i = *i + (z == 0 ? 1 : z);
}

}

int main() { ll t, N, i = 0;

srand(time(NULL));
printf("Enter the total number of frames: ");
scanf("%lld", &t);
printf("Enter window size: ");
scanf("%lld", &N);
transmission(&i, N, t);
printf("Total no. of frames which were sent and received are: %lld\n",
i);
return 0;

}
```

4) DIJSKTRA

```
#include <stdio.h>

int main() {

   int a[5][5], path[10][10];

   int p, n = 5;

   int i, j;

   int cost[10], minCost, index;

   printf("Enter the cost matrix (5x5):\n");

   for (i = 0; i < n; i++) {

      for (j = 0; j < n; j++) {

         scanf("%d", &a[i][j]);

      }

   }
```

```
printf("Enter number of paths: ");
scanf("%d", &p);
printf("Enter each path as a sequence of 5 nodes (0–4):\n");
for (i = 0; i < p; i++) {
   for (j = 0; j < n; j++) {
      scanf("%d", &path[i][j]);
   }
}
for (i = 0; i < p; i++) {
   cost[i] = 0;
   for (j = 0; j < n - 1; j++) {
      cost[i] += a[path[i][j]][path[i][j + 1]];
   }
}
minCost = cost[0];
index = 0;
for (i = 1; i < p; i++) {
   if (cost[i] < minCost) {
      minCost = cost[i];
      index = i;
   }
}
printf("\nMinimum cost: %d\n", minCost);
```

```
    printf("Minimum cost path: ");
    for (j = 0; j < n; j++) {
        printf("%d ", path[index][j]);
    }
    printf("\n");
    return 0;
}
```

5) SUBNET OF HOSTS

```
#include <stdio.h>
int a[10][10], n;
void adj(int k);
int main()
{
    int i, j, root;
    printf("Enter number of nodes: ");
    scanf("%d", &n);
    printf("Enter the adjacency matrix:\n");
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            printf("Enter connection of %d -> %d: ", i, j);
            scanf("%d", &a[i][j]);
        }
    }
    printf("Enter root node: ");
    scanf("%d", &root);
    adj(root);
```

```
    return 0;
}
void adj(int k)
{
    int i, j;
    printf("\nAdjacent nodes of root node %d:\n", k);
    printf("Outgoing edges:\n");
    for (j = 1; j <= n; j++) {
        if (a[k][j] == 1) {
            printf("%d ", j);
        }
    }
    printf("\nIncoming edges:\n");
    for (i = 1; i <= n; i++) {
        if (a[i][k] == 1) {
            printf("%d ", i);
        }
    }
    printf("\n");
}
```

6) DISTANCE VECTOR ROUTING

```
#include <stdio.h>
struct node
{
    unsigned dist[20];
    unsigned from[20];
}
```

```
rt[20];
int main()
{
   int costmat[20][20];
   int nodes, i, j, k, count = 0;
   printf("\nEnter the number of nodes: ");
   scanf("%d", &nodes);
   printf("\nEnter the cost matrix:\n");
   for (i = 0; i < nodes; i++) {
      for (j = 0; j < nodes; j++) {
         scanf("%d", &costmat[i][j]);
         if (i == j)
            costmat[i][j] = 0;
         rt[i].dist[j] = costmat[i][j];
         rt[i].from[j] = j;
      }
   }
   do {
      count = 0;
      for (i = 0; i < nodes; i++) {
         for (j = 0; j < nodes; j++) {
            for (k = 0; k < nodes; k++) {

               if (rt[i].dist[j] > costmat[i][k] + rt[k].dist[j]) {
                  rt[i].dist[j] = costmat[i][k] + rt[k].dist[j];
                  rt[i].from[j] = k;
                  count++;
```

```
                }


            }
          }
        }
    } while (count != 0);
    printf("\n--- Routing Tables ---\n");
    for (i = 0; i < nodes; i++) {
        printf("\nRouter %d:\n", i);
        for (j = 0; j < nodes; j++) {
            printf("Node %d via %d distance = %d\n",
                j, rt[i].from[j], rt[i].dist[j]);
        }
    }
    return 0;
}
```

7) Data encryption decryption

```
#include <stdio.h>
int main()
{
    int i, x;
    char str[100];
    printf("\nPlease enter a string:\n");
    scanf("%s", str);
    printf("\nPlease choose from the following options:\n");
    printf("1 = Encrypt the string\n");
```

```
    printf("2 = Decrypt the string\n");
    scanf("%d", &x);
    switch(x)
    {
        case 1:
            for(i = 0; i < 100 && str[i] != '\0'; i++)
            {
                str[i] = str[i] + 3;
            }
            printf("\nEncrypted string: %s\n", str);
            break;
        case 2:
            for(i = 0; i < 100 && str[i] != '\0'; i++)
            {
                str[i] = str[i] - 3;
            }
            printf("\nDecrypted string: %s\n", str);
            break;
        default:
            printf("\nError\n");
    }
    return 0;
}
```

8) Leaky bucket

```
#include <stdio.h>
```

int main() { int no_of_queries, storage, output_pkt_size; int input_pkt_size, bucket_size, size_left;

```
storage = 0;

no_of_queries = 4;
bucket_size = 10;

input_pkt_size = 4;
output_pkt_size = 1;

for (int i = 0; i < no_of_queries; i++)
{
    size_left = bucket_size - storage;

    if (input_pkt_size <= size_left)
    {
        storage += input_pkt_size;
    }
    else
    {
        printf("Packet loss: %d\n", input_pkt_size);
    }

    printf("Buffer size: %d out of bucket size: %d\n", storage,
bucket_size);

    if (storage >= output_pkt_size)
    {
        storage -= output_pkt_size;
    }
}

return 0;


}
```

9) Frame sorting

```
#include <stdio.h>
struct frame
{
   int num;
   char str[20];
};
struct frame arr[10];
int n;
void sort()
{
   int i, j;
   struct frame temp;
   for (i = 0; i < n - 1; i++)
   {
      for (j = 0; j < n - i - 1; j++)
      {
         if (arr[j].num > arr[j + 1].num)
         {
            temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
         }
      }
   }
}
int main()
{
```

```
    int i;
    printf("Enter the number of frames:\n");
    scanf("%d", &n);
    printf("Enter the frame sequence number and frame contents:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d %s", &arr[i].num, arr[i].str);
    }
    sort();
    printf("\nThe frames in sequence are:\n");
    for (i = 0; i < n; i++)
    {
        printf("%d\t%s\n", arr[i].num, arr[i].str);
    }
    return 0;
}
```